things you'll have to deal with in an event-driven environment is the tension between asynchronous code and the desire to structure tests synchronously.

10.4. Testing the asynchronous

CoffeeScript programs, usually run in the browser or on Node.js, are usually event driven, and I/O is almost always asynchronous. Why is this a problem? Well, consider the following test structure:

```
fact 'the tracking application tracks a user mouse click', ->
  options = {}
  tracking = new Tracking options, http
  tracking.start ->
  fred.visit '/page/1'
  assert.equal tracking.total(), 1
```

The `start` method of the `tracking` object expects a callback function that will be invoked when it's finished. But the test invokes the `visit` method of the `fred` object and asserts the result of invoking the `total` method on the `tracking` object immediately, without bothering to wait for the `start` method to finish. If the `start` method hasn't finished by the time the assertion occurs, the test will fail.

There are three related techniques for dealing with asynchronous I/O in tests: live with it, remove it, or expect it.

10.4.1. Live with it

One technique for testing asynchronous programs is to just reflect the asynchronous nature in your tests. To do this, add assertions as part of a callback to the function you're testing: